Faculty of Engineering

Department of Informatics Engineering

Software Engineering and Information Systems

Artificial Intelligence Engineering and Data Science

Drug management platform

A junior project report - submitted to complete the requirements for obtaining a bachelor's degree in informatics engineering -Artificial Intelligence Engineering and Data Science

Prepared by

Abdulkarim Salaymeh

Supervised by

Eng. Maher Sarem

الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

كليـــــــــــة الهندســـــــــــــة
قســـــم الهندســـة المعلوماتيـــة
هندسة البرمجيات ونظم المعلومات
هندسة الذكاء الصنعي وعلوم البيانات

منصة ادارة الادوية

تقرير للمشروع الفصلي قدّم استكمالاً لمتطلبات الحصول على درجة
البكالوريوس في هندسة المعلوماتية - هندسة الذكاء الصنعي وعلوم البيانات

إعداد

عبدالكريم سلايمة

اشراف

المهندس. ماهر صارم

Table of Contents

Chapter 1: General Introduction to the Medication Management and Dose Reminder System

This chapter provides a comprehensive breakdown of the project in terms of its background, the real-world problem it addresses, its primary goals, practical scope, anticipated scientific contributions, and the research methodology utilized to guide the documentation. It establishes the conceptual foundation of the report, connecting concrete patient needs with the software solution engineered to fulfill them.

1.1 Project Overview and Background

The core focus of this project is developing a centralized web application designed for comprehensive medication tracking and automated dose scheduling. It enables patients to safely compile a personalized digital locker of their current prescriptions, log the exact real-time quantities of medication on hand, specify strict minimum threshold metrics to spark automatic restocking alerts, and design dynamic dosage tables. These multi-dimensional tables securely track exact recurrence schedules across specific calendar days, repeating temporal frequencies, and micro-timed intake hours.

When a scheduled medication window arrives, the system background thread automatically triggers a prompt notification, prompting the user via an active web portal overlay. Users can seamlessly mark the scheduled event as either 'Confirmed/Taken' or 'Missed/Skipped' directly from the graphical user interface. The technical implementation leverages Flask as the primary web application factory framework, fortified by a stateless authentication architecture powered by JSON Web Tokens (JWT). Persistent system data is mapped to a robust MySQL server instance abstraction utilizing the SQLAlchemy Object-Relational Mapper (ORM). Simultaneously, a multi-threaded scheduling execution layer continuously checks active tables to accurately calculate micro-timed windows, fire system-wide alerts, and process lower-bound threshold metrics.

The foundational significance of this implementation lies in rectifying a pervasive, high-risk real-world issue: the profound therapeutic complications stemming from low patient compliance and inconsistent adherence due to complex, multi-medication routines. It represents a highly structured software engineering blueprint that demonstrates seamless programmatic convergence across dynamic inventory auditing, background job scheduling, synchronous notification routing, and automated compliance auditing.

Component	Functional Role	System Impact
Authentication (JWT)	Validates patient or administrator identity securely via signed stateless tokens	Establishes bulletproof security context and absolute role separation
Medication Catalog	Allows administrators to add, modify, activate, or systematically	Grants full database control over the system-wide pharmaceutical

	deprecate entries	index
Patient Medication Vault	Stores dynamic inventory metrics, personal schedules, and lower boundaries	Reflects the actual current usage states and physical supply metrics
Dosage Scheduling Matrix	Maps out precision recurrence rules, calendar tracking, and exact timelines	Maintains absolute persistence of the multi-tiered clinical treatment plan
Notifications & Adherence Logs	Handles prompt reminder generation and writes unalterable historical logs	Enables precision auditing of real-world patient therapeutic compliance

1.2 Problem Statement, Practical & Academic Significance

The core problem tackled by this architecture is the widespread absence of dependable, automated, and interconnected self-management tools to help patients follow precise therapeutic routines, particularly when dealing with complex, multi-drug regimens. Traditional manual methods rely heavily on personal cognitive recall, static written diaries, or disconnected smartphone clock alarms. This fragmentation significantly escalates the clinical probability of critical dosing oversights, dangerous therapeutic delays, or accidental double-dosing.

Practically, systematic non-adherence drastically compromises clinical efficacy, accelerates disease progression, increases avoidable emergency readmissions, and places an exhausting tracking burden on family members and healthcare providers. Academically, this dilemma serves as an ideal software engineering showcase to explore how complex healthcare needs translate into decoupled, highly structured multi-tiered architectures.

Problem Dimension	Detailed Description	Direct System Impact
Dosing Oversights	Patients completely forget to take critical medication due to cognitive overload	Severe disruption of therapeutic blood-plasma concentrations
Routine Complexity	Managing multiple drugs simultaneously leads to dangerous administration errors	Elevated risk of cross-medication confusion and accidental toxicity
Absence of Auditing	No immutable historical log exists to	Prevents physicians from objectively

	verify past intake events	evaluating therapeutic efficacy
Supply Depletion	Physical medication runs out unexpectedly, causing sudden treatment gaps	Abrupt therapy interruption before a pharmacy refill is secured

1.3 Project Objectives

The fundamental objectives of this project were explicitly mapped out to ensure complete testability and verifiable implementation within clear system constraints. The primary goal centers on engineering a fully integrated web platform that streamlines medication management and dose reminders, supported by distinct sub-objectives:

Objective Category	Specific Technical Objective	Measurable Success Indicator
Primary Metric	Build an interconnected web platform for medication oversight and scheduling	Flawless end-to-end execution from drug entry to active notification logging
Sub-Objective	Provide comprehensive user self-service for inventory tracking and limit mapping	Successful modification of private records via the user dashboard
Sub-Objective	Implement flexible scheduling logic across specific days and time matrices	Active database generation of complex relational rules per medication
Sub-Objective	Deploy an automated background engine to fire precision reminder notifications	System-wide delivery of instant pop-ups matching exact schedule slots
Sub-Objective	Establish a secure, unalterable historical ledger for intake auditing	Accurate archival of compliance states accessible via user histories
Sub-Objective	Enforce strict Role-Based Access Control (RBAC) boundaries	Absolute HTTP 403 authorization rejections against cross-role route requests

1.4 Project Scope, Constraints & Assumptions

The structural scope encompasses a secure web application environment operating across dual access roles (Patient and Administrator). It natively manages user onboarding, master catalog administration, user-specific inventory tracking, relational dosage scheduling, real-time reminder delivery, and immutable compliance logging. The underlying database engine maintains tight referential integrity across accounts, drug indexes, live schedules, notification queues, and historical logs.

Conversely, the scope explicitly excludes automated medical diagnosis, prescription suggestions, telemetry interfacing with external wearable biometric hardware, or direct integration with regional hospital networks. The system does not employ predictive machine learning heuristics; it functions strictly on deterministic, rule-based scheduling algorithms and periodic background verification loops.

Scope/Constraint Element	Technical Details	Impact on System Design
Architectural Scope	Decoupled web portal managing patient profiles, catalogs, and alerts	Requires clean separation of concerns via an explicit multi-tiered layout
Technical Constraint	Relies entirely on persistent servers and manual user inventory inputs	Demands strict server-side validation against corrupted inputs
Security Constraint	Enforces stateful token blocklisting upon explicit session termination	Requires a high-speed database registry for invalidated token hashes
System Assumption	Assumes end-users possess basic web literacy and interface familiarity	Prioritizes clean, straightforward, and accessible graphical user interfaces
Operational Assumption	Assumes user-provided timezones and inventory values are correct	Enforces structural validation before saving records to the database

Chapter 2: Project Management and Development Methodology

This chapter details the methodology utilized to guide the project from initial conceptualization to a fully validated production-ready state. It documents how abstract ideas transformed into structured, measurable software deliverables. Given the application's complex dependencies across real-time background jobs, inventory auditing, stateless security layers, and historical logging, selecting an adaptive development approach was a foundational requirement.

2.1 Selection and Justification of Development Methodology

The project embraced an iterative hybrid development framework, combining flexible Agile/Scrum cycles with the rigorous documentation standards typical of academic software engineering. This approach was chosen because system specifications emerged progressively during building; complex components like real-time alert routing, interactive intake confirmation logic, lower-bound safety triggers, and administrative overrides required repeated cycles of prototyping and user testing.

The iterative lifecycle allows each architectural component to be built independently, rigorously tested, and seamlessly integrated into the broader environment. This capability is vital because the core modules are tightly coupled: adding a new prescription alters the active calendar tables, which changes the execution queue of the background scheduler, which drives the real-time notification engine, which updates physical stock volumes upon user validation.

Methodology	Key Strengths	Primary Weaknesses	Justification for Project Selection
Waterfall	Highly predictable structure with rigid phase boundaries	Extremely difficult to adapt to mid-cycle requirement adjustments	Rejected due to evolving scheduling logic and dynamic feature design
Agile / Scrum	Exceptional flexibility, rapid prototyping, and frequent testing	Demands constant team alignment and rigid tracking discipline	Adopted for the technical build to enable incremental component deployment
Hybrid Approach	Balances rapid, flexible development with structured documentation	Requires careful management to maintain process equilibrium	Selected as the ideal framework for this rigorous academic engineering project

2.2 Project Timeline (Gantt Plan)

System construction followed a strict weekly schedule to ensure balanced progress across requirements analysis, architectural modeling, back-end coding, front-end implementation, rigorous testing, and final documentation. This early schedule prevented common bottlenecks near deadlines.

Development Phase	Timeframe	Core Architectural Deliverables	Strategic Operational Notes
Requirements Analysis	Weeks 1–2	Software Requirements Specification (SRS) & role definitions	Deep analysis of scheduling rules and edge-case exceptions
Architectural Design	Weeks 2–3	Database schema, entity-relationship diagrams, API blueprints	Prioritized highly scalable, decoupled software layers
Back-End Engineering	Weeks 3–5	Robust services, database repositories, and secure REST endpoints	Leveraged Flask factories and SQLAlchemy data mappers
Scheduler & Alerts	Weeks 5–6	Background worker execution loop and threshold triggers	Critical focus on accurate timezone calculations and precision timing
Front-End Integration	Weeks 4–6	Responsive patient dashboards and admin consoles	Connected client-side AJAX calls to secure REST APIs
Validation & Debugging	Weeks 6–7	Comprehensive integration testing and end-to-end flow audits	Evaluated edge-case scheduling scenarios and rapid token cancellation
Final Documentation	Weeks 7–8	Rigorous academic project report compilation	Aggregated architecture diagrams and verified traceability matrices

2.3 Risk Management

Risk management for this platform extended beyond standard server failures to address complex operational risks, including data sync issues, scheduling drifts, or unvalidated privilege escalation. A proactive risk register was established to mitigate these threats.

Identified Risk	Probability	Impact	Proactive Engineering Mitigation Strategy
Shifting System Requirements	Medium	High	Maintained highly flexible modular layers and updated documentation iteratively
Timezone Calculations Drift	Medium	High	Standardized backend tracking on UTC timestamps and applied client-side rendering offsets
Database Connection Loss	Low	High	Implemented robust connection pooling and encapsulated configurations in secure environment files
Corrupted User Form Inputs	High	Medium	Enforced rigid multi-stage validation checks across both UI and server API layers
Privilege Escalation Attacks	Low	High	Fortified backend endpoints with cryptographic token claims checks and strict RBAC decorators
Documentation Bottlenecks	Medium	Medium	Wrote engineering logs synchronously alongside feature code to ensure accuracy

Chapter 3: Literature Review and Previous Studies

This chapter establishes the scientific foundation of the project by surveying previous research in digital medication management and adherence tracking. It explores current technologies, conducts a comparative analysis of existing platforms, and pinpoints the structural limitations this system addresses. It connects the core problem with the engineered solution detailed in subsequent design chapters.

3.1 Previous Studies in Medication Reminder Systems

Academic research in this area typically focuses on three primary objectives: improving patient adherence to therapy, enhancing interface accessibility, and reducing tracking burdens on caregivers. Existing solutions generally split into three form factors: native mobile apps, automated SMS alerts, or specialized electronic pillboxes equipped with custom telemetry hardware.

Year	Author / Study Reference	Core Concept Overview	Primary Technical Findings	Identified Weaknesses	Direct Structural Relevance
2013	Dayer et al.	Evaluated mobile app alerts for centralized prescription management	Proved apps improve patient compliance by consolidating tracking	Completely ignored inventory auditing or unalterable compliance histories	Validates mobile access, extended here to a robust web framework
2016	Thakkar et al.	Analyzed structured text alerts for chronic disease patients	Proved automated SMS messaging drives high therapy adherence	Lacked multi-drug scheduling matrix or inventory tracking	Confirms reminder value, extended here to real-time UI pop-ups
2020	Peng et al.	Evaluated mobile health software for chronic illness routines	Confirmed software tools significantly boost adherence rates	Noted widespread data vulnerabilities and poor cross-layer security	Highlights the critical need for robust layer separation and security
2021	Abu-El-Noor et al.	Tested customized hypertension	Proved app usage leads to measurably	Lacked a centralized admin	Reaffirms automated reminders,

		tracking software applications	superior clinical compliance	dashboard and RBAC controls	extended here to multi-role admin controls
2023	Manyazewal et al.	Evaluated custom telemetry hardware containers for tuberculosis care	Reported exceptional satisfaction with hardware tracking solutions	Tied strictly to single-drug devices; lacked cloud-scale flexibility	Proves adherence relies on tracking; extended to digital web models
2023	Hartch et al.	Deployed low-overhead alert apps in resource-constrained areas	Proved digital alerts are highly viable in complex healthcare environments	Lacked enterprise-grade role separation or granular access controls	Supports lightweight design running on explicit, low-overhead workflows

3.2 Modern Technologies and Frameworks in the Field

Modern digital health applications require highly stable, secure backends, robust database mapping, stateless authentication mechanisms, and low-latency task schedulers. This application achieves these requirements via an explicit architecture: Flask acts as the streamlined web factory server, SQLAlchemy serves as the data access layer ORM, Flask-JWT-Extended enforces granular session authorization, APScheduler drives the real-time background threads, and MySQL manages robust, relational data persistence.

3.3 Comparative Analysis of Existing Systems & Products

To highlight the practical necessity of the proposed platform, it was benchmarked against alternative approaches: generic alert apps, dedicated hardware pillboxes (MERM), and specialized single-disease tracking software. This matrix demonstrates that while existing solutions manage basic alerts well, they rarely integrate inventory auditing, historical compliance logging, and administrative access controls into a unified application.

Platform Category	Reminder Mechanism	Historical Compliance Log	Inventory Auditing	Multi-Role Admin Console	Primary Functional Bottleneck
Generic Alert	Push notifications or	Rudimentary or entirely manual	Generally	Not supported	Provides basic text alerts but

Apps	simple SMS lines	inputs	unsupported		fails to track supply levels
MERM Devices	Physical beep sounds upon opening box	Automatically tracked on device flash memory	Not supported	Not supported	Highly effective telemetry but requires costly, specialized hardware
Single-Disease Apps	Rigid time schedules for target illnesses	Fully supported for targeted routines	Extremely limited	Not supported	Excellent clinical focus but lacks multi-drug flexibility
Proposed Platform	Real-time web pop-ups driven by background threads	Fully automated, immutable historical ledger	Supported with automated low-stock warnings	Centralized, secure administration console	Provides complete unification of inventory, alerts, logs, and roles

3.4 Research Gap and Proposed Innovation

The primary product design gap lies in the fragmentation of existing software tools: most focus exclusively on basic clock alerts, completely ignoring live inventory levels, unalterable compliance auditing, or multi-role administration. The proposed platform bridges this gap by unifying these components into a single web workspace. It automates inventory updates by deducting stock values immediately upon intake confirmation, alerts users before supplies run out, and maintains historical data for accurate clinical evaluation, all within a highly secure, role-separated framework.

3.5 Theoretical Framework and Reference Studies

The application is built upon three foundational software concepts. First, Medication Adherence—the quantifiable measurement of patient compliance with a prescribed routine. Second, Time-based Reminders—decoupling time calculation from user memory via automated background threads. Third, State Tracking—guiding data objects through strict life cycles (e.g., Scheduled, Fired, Confirmed, Missed). In terms of software design, it strictly follows the Separation of Concerns (SoC) principle by decoupling routers, business logic, repositories, data models, and background tasks.

3.6 Modern and Future Trends in the Field

Current trends in healthcare software emphasize clean, user-friendly interfaces, automated data capturing, and proactive alert mechanisms. Future iterations of this system could easily integrate

behavioral compliance analytics or external database APIs. However, the current release focuses entirely on providing a highly secure, automated, and rule-driven infrastructure to ensure absolute operational reliability.

Chapter 4: Requirements Modeling and Analysis

This chapter translates user requirements into precise technical specifications. It outlines the feasibility study, requirements gathering, stakeholder analysis, functional/non-functional criteria, prioritization matrices, UML diagrams, and preliminary test plans. This structured engineering analysis follows the formal guidelines required for comprehensive academic project documentation.

4.1 Feasibility Study

Technically, the project is highly feasible, relying on stable, well-supported open-source components: Flask for the application engine, SQLAlchemy for ORM mapping, JWT for security, APScheduler for background tasks, and MySQL for reliable storage. Economically, development overhead remains low, requiring no specialized hardware or licensing fees, making it highly suitable for standard cloud hosting environment footprints. Operationally, the application features an intuitive web interface tailored for seamless navigation by patients of varying digital literacy levels.

4.2 Requirements Gathering

System requirements were gathered by performing a comprehensive code-level analysis of the core application modules, data models, route maps, service functions, and server-side validation forms. This approach ensured that the requirements documentation perfectly matches the actual system implementation, avoiding abstract or unverified project assumptions.

Requirement Source	Technical Collection Description	Direct Architectural Impact	Direct Validation Impact
Codebase Review	Extracted functional capabilities directly from implemented modules	Mapped out back-end services, routes, and schemas	Drove the creation of targeted end-to-end integration tests
Database Profiling	Analyzed database constraints, foreign keys, and indexes	Enforced relational rules across core data models	Verified strict database constraints and schema boundaries
Interface Auditing	Analyzed client-side view routers and data flow mechanics	Aligned backend REST APIs with front-end UI needs	Guided the design of functional user interface test plans
Log Engine Analysis	Reviewed background job schedules and log generation scripts	Designed the core background task worker execution ring	Validated system timing precision and background task execution

4.3 Stakeholders and Needs Analysis

The application addresses the specific requirements of four core operational roles, ensuring a balanced design that serves end-users, system administrators, database engines, and background tasks simultaneously.

Stakeholder Role	Operational System Focus	Core Technical Requirements	Engineered Solution Delivery
Patient	Primary system end-user	Requires clear scheduling, reliable alerts, inventory audits, and low-stock warnings	Delivers custom web portals, live alerts, unalterable history logs, and stock tracking
Administrator	System operations manager	Requires global catalog control, user account oversight, and custom alert routing	Provides custom admin consoles with global data controls and manual overrides
Background Task Engine	Automated system worker	Demands low-overhead loops, accurate timezone tracking, and rapid job execution	Deploys multi-threaded background loops targeting live schedule records
Database Engine	Data persistence layer	Demands absolute data consistency, zero record duplication, and fast queries	Enforces rigid relational constraints, foreign keys, and indexed data tables

4.4 Functional Requirements

The platform's functional requirements define its explicit programmatic capabilities, with every requirement directly supported by targeted back-end service functions and secure REST API endpoints.

ID Code	Requirement Technical Specification	Primary Role Beneficiary	Back-End Implementation Mapping
FR-01	Users must be able to register accounts and securely log in via	Patient	AuthService routing through secure authentication endpoints

	protected interfaces		
FR-02	Administrators must be able to securely authenticate into dedicated oversight consoles	Administrator	AuthService routing through dedicated admin login paths
FR-03	The system must display an index of medications with built-in search filters and pagination	Patient / Admin	MedicationService querying catalog records with built-in page offsets
FR-04	Patients must be able to link medications to their accounts with stock counts and alerts	Patient	PatientMedication entity mapping inventory parameters to accounts
FR-05	Patients must be able to modify, pause, or reactivate linked medication records at any time	Patient	PatientMedication state machine managing active/paused flags
FR-06	The system must support complex scheduling rules linking medication records to active times	Patient	DosageSchedule mapping database rules to specific timing entries
FR-07	The system must automatically fire targeted pop-up notifications when schedule slots arrive	Patient	Background scheduler checking database entries and firing UI alerts
FR-08	Patients must be able to mark active alerts as 'Taken' or 'Missed' directly from the interface	Patient	DoseLog tracking user responses to active notification alerts
FR-09	The system must automatically deduct medication stock values	Patient	DoseService inventory logic processing volume

	upon intake confirmation		updates in real-time
FR-10	The system must alert users immediately when stock counts cross lower safety boundaries	Patient	Background thread scanning stock metrics and firing warning notifications
FR-11	Administrators must be able to route manual warning notifications directly to target users	Administrator	AdminService routing custom message objects straight to user notification feeds
FR-12	The system must securely store all alerts, compliance logs, and read states indefinitely	Patient / Admin	DoseLog and Notification data tables maintaining permanent records

4.5 Non-Functional Requirements

Non-functional requirements define the application's core quality boundaries, ensuring optimal performance, security, reliability, and long-term maintainability.

Category	Target Metric	Technical Description	Verification Method
Performance	API Latency	Standard REST API endpoints must return data payloads within acceptable limits	Benchmarked endpoint responses under heavy query conditions
Security	RBAC Security	Secure paths must reject unauthorized requests missing signature payloads	Tested route access with missing, corrupt, or expired token headers
Reliability	Token Safety	Invalidated token signatures must be rejected instantly by security layers	Attempted system access utilizing explicitly blocklisted tokens
Usability	UI Simplicity	User layouts must be intuitive, minimizing clicks	Conducted UI workflow evaluations across multi-

		for core actions	stage scheduling paths
Maintainability	Decoupled Layout	The application must maintain clear separation between services and storage	Performed structural reviews of module boundaries and dependencies
Scalability	Background Tasks	Background loops must execute concurrently without locking front-end UI tasks	Verified scheduler thread separation under heavy background loads

4.6 Prioritization and MoSCoW Management Mechanism

The MoSCoW framework was applied to prioritize development tasks, ensuring that core architectural components were completed first, while segregating non-essential enhancements.

Priority	Target System Features	Justification
Must Have	Secure login, profile linking, scheduling tables, notifications, and stock auditing	The baseline features required to make the application functionally viable
Should Have	Global catalog searches, dynamic classification filters, and historical compliance charts	Significantly improves usability but does not break core workflows if omitted
Could Have	Advanced interface styling, additional tooltips, and customizable notification feeds	Provides non-essential styling polish that can be deferred to later cycles
Won't Have	Wearable hardware telemetry or automated, predictive medical diagnostic logic	Explicitly falls outside the scope of this project release cycle

4.7 UML Diagrams for Requirements

System requirements are structurally detailed via standard UML behaviors, capturing use cases, execution timelines, and cross-layer data flows cleanly.

[Use Case Specification Table]

Use Case Title	Primary Actor	Standard Execution Flow	Alternative Path Handling	Resulting System State
Manage Vault	Patient	User navigates workspace, adds a drug profile, and sets limits	Rejects submission if the target drug is missing or deprecated	Applies new inventory record to the user's private vault
Set Schedule	Patient	User selects a linked drug and configures calendar tracking rules	Rejects input if schedule boundaries contain invalid date values	Generates an active dosage schedule entry in the database
Audit Dose	Patient	User interacts with a live alert pop-up, marking it taken or missed	Blocks confirmation if the target alert index is invalid or missing	Writes a permanent entry to compliance history logs
Manage Catalog	Administrator	Admin accesses the control layout to add, alter, or pause catalog rows	Blocks changes if required input fields are blank or duplicated	Saves global catalog modifications to the master records
Manual Warning	Administrator	Admin targets a specific user account and routes a custom alert message	Aborts delivery if the target account identifier does not exist	Pushes custom warning message to the target user's feed

4.8 Requirements Traceability Matrix

To ensure total accountability, this matrix traces every functional requirement from its analytical definition down to its specific backend database structures and validation scripts.

Functional Code	Use Case Context Mapping	Database Table / Service Mapping	Initial Verification Strategy
FR-01	User Profile Management	AuthService / Patient model	Execute automated user onboarding and check

			login return payloads
FR-03	Medication Discovery	MedicationService / Medication model	Query database catalog indexes using complex search strings
FR-04	Inventory Onboarding	PatientMedication model	Link a medication profile to a test account with custom safety bounds
FR-06	Matrix Scheduling	DosageSchedule / DoseTime models	Generate structured day/time recurrence rows for a linked drug
FR-07	Alert Infrastructure	Background Worker / NotificationService	Trigger automated background check loops and verify alert delivery
FR-08	Compliance Tracking	DoseService / DoseLog model	Acknowledge an active notification pop-up and verify history logging
FR-10	Stock Depletion Tracking	NotificationService logic	Reduce inventory counts to zero and verify low-stock warning generation

4.9 Data Schema and Core Relationships

The underlying relational database schema enforces strict referential integrity across all system data. Accounts link dynamically to medication profiles, scheduling models track precise calendar windows, and history tables maintain unalterable compliance logs, as visualized in the system's structural relationship maps.

4.10 Preliminary Test Cases

Preliminary functional test cases were designed during the requirement modeling phase to establish clear validation targets before starting back-end coding.

Test Case ID	Target Evaluation Context	Provided Form Inputs	Expected Functional Result	Initial Status

TC-01	User Profile Onboarding	Valid names, unlinked email values, and strong passwords	Generates database profile and returns signed token payload	PASS
TC-02	Inventory Link Generation	Valid database drug identifier and positive stock metrics	Generates new row inside the PatientMedication table	PASS
TC-03	Schedule Matrix Mapping	Valid linked record code, day arrays, and clean time values	Saves active data objects to scheduling tables	PASS
TC-04	Premature Dosing Audits	Future timestamp markers and un-fired notification IDs	API layer blocks request with an explicit validation error	PASS
TC-05	Alert Intake Processing	Active alert identifier mapping to a valid pending notification	Writes compliance record and decreases inventory count	PASS
TC-06	Boundary Depletion Audits	Simulated stock adjustments falling below lower boundaries	Background thread automatically fires a low-stock alert	PASS

Chapter 5: Architectural Design of the System

This chapter explains the system's software architecture. It details how abstract requirements translate into a highly structured, maintainable, multi-tiered architecture, defining decoupled functional layers, relational schema boundaries, RESTful API specifications, role guards, and multi-threaded scheduling workflows.

5.1 High-Level Design & Pattern Used

The platform implements a highly structured Layered Architecture style, separating presentation views, controllers, core business logic, and database repositories. This decoupling minimizes cross-module dependencies, making the codebase easier to test, maintain, and scale.

The web routing layer relies on Flask Blueprints to isolate endpoints by context. Incoming HTTP requests are intercepted by input validators, processed through core business services, and persisted via repositories utilizing SQLAlchemy ORM mappers to execute transactions against the MySQL database server instance safely.

Architectural Layer	Primary Technical Responsibility	Project Implementation Examples
Routes / Blueprints	Intercepts HTTP requests, extracts parameters, and handles JSON mapping	auth_routes, medication_routes, dose_routes, admin_routes
Input Validators	Enforces rigid structural data sanitization before logic execution	auth_validators, medication_validators, dose_validators packages
Business Services	Executes core multi-tiered transactional and operational system rules	AuthService, MedicationService, DoseService, NotificationService
Data Repositories	Encapsulates complex database queries and isolates storage transactions	PatientRepository, DoseLogRepository, NotificationRepository
SQLAlchemy Models	Defines relational entities, field data constraints, and database maps	Patient, Medication, DosageSchedule, DoseTime, DoseLog entities
Background Workers	Runs continuous, asynchronous timing ring execution loops	Multi-threaded reminder_scheduler tasks built on top of APScheduler

5.2 Database Design

The database schema relies on a highly structured relational configuration to ensure absolute data consistency. The primary table mappings, unique operational constraint rules, and database fields are detailed in the structural tables below.

Table Name	Core Schema Attributes	Primary Key	Relational Mapping Context	Data Persistence Role
patient	patient_id, full_name, email, password_hash, phone, status	patient_id	1-to-many links with vault records and account alerts	Persists core patient account profiles
admin	admin_id, name, email, password_hash, security_clearance, status	admin_id	1-to-many links with system-wide warning logs	Persists system administrator accounts
medication	medication_id, name, category, form, strength, is_active	medication_id	1-to-many links with user-specific vault entries	Maintains global pharmaceutical master catalog
patient_medication	patient_med_id, patient_id, medication_id, current_quantity, min_threshold	patient_med_id	Many-to-1 links tracking patient-to-medication records	Audits active inventory and safety limits
dosage_schedule	schedule_id, patient_med_id, start_date, end_date, is_continuous	schedule_id	1-to-many links mapping to specific days and times	Persists complex prescription timing routines
dosage_schedule_day	schedule_day_id, schedule_id, day_of_week	schedule_day_id	Many-to-1 links mapping recurrence rules to weekdays	Tracks weekly scheduling calendars
dose_time	dose_time_id, schedule_id,	dose_time_id	Many-to-1 links mapping micro-	Tracks daily intake timing variables

	dose_period, dose_time, dose_amount, dose_unit		timed dosing slots	
dose_log	dose_log_id, patient_med_id, dose_time_id, scheduled_time, taken_time, status	dose_log_id	Many-to-1 links linking history entries to vault logs	Maintains an unalterable compliance ledger
notification	notification_id, patient_id, patient_med_id, type, message, status	notification_id	Many-to-1 links connecting alerts to target users	Tracks system-wide alert dispatch histories
blacklisted_token	id, jti, token_type, expires_at, created_at	id	Stands completely independent of operational data entities	Tracks revoked token signature identifiers

Target Constraint Name	Table Location	Programmatic Functional Purpose	Direct Data Consistency Impact
Unique(patient_id, medication_id)	patient_medication	Prevents duplicate entries of the same drug profile in a user's vault	Enforces a single, clean tracking file per drug
Unique(schedule_id, day_of_week)	dosage_schedule_day	Prevents weekday duplication within an active scheduling group	Ensures clean structural calendar parameters
Unique(schedule_id, dose_period)	dose_time	Restricts daily schedules to a single action entry per target period	Separates morning, afternoon, and evening slots
Unique(type, dose_time_id, scheduled_for)	notification	Prevents duplicate alerts from firing for the same scheduling event	Eliminates user dashboard alert clutter

Unique(patient_med_id, dose_time_id, scheduled_time)	dose_log	Prevents duplicate status logs for the same scheduling window	Protects compliance ledger accuracy

5.3 API Design

The communication architecture strictly adheres to RESTful API conventions, utilizing structured JSON payloads for clean data exchange. All sensitive backend endpoints are protected behind secure cryptographic role validators.

API Target Endpoint Route	HTTP Method	Required Payload Variables	Functional Operation Description	RBAC Boundary Guard
/api/auth/register	POST	full_name, email, password, phone	Registers a new patient account and issues an access token	Accessible to all clients
/api/auth/login	POST	email, password	Validates patient credentials and signs an access token	Accessible to all clients
/api/auth/admin/login	POST	email, password	Validates administrator credentials and returns an admin session token	Accessible to all clients
/api/medications/my	GET	page, per_page query params	Fetches pagination records from the user's private vault	Patient Role Required
/api/medications/my	POST	medication_id, current_quantity, min_threshold	Links a global drug entry to the patient's vault with stock limits	Patient Role Required
/api/medications/my/<id>/schedule	POST	start_date, end_date, days,	Generates complex scheduling rules	Patient Role Required

		times arrays	and daily intake time matrices	
/api/doses/my/<id>/confirm	POST	dose_time_id, scheduled_date	Confirms medication intake, updates stock, and writes history logs	Patient Role Required
/api/doses/my/<id>/missed	POST	dose_time_id, scheduled_date	Logs a skipped medication event to the historical compliance ledger	Patient Role Required
/api/notifications/my	GET	page, per_page query parameters	Fetches active alerts and updates the user notification dashboard	Patient Role Required
/api/notifications/my/<id>/read	PATCH	No payload body required	Marks a targeted notification alert as read on the dashboard	Patient Role Required
/api/admin/patients	GET	q search strings, pagination filters	Provides global access to patient records for administrative auditing	Admin Role Required
/api/admin/medications	POST	name, category, form, strength fields	Saves a new pharmaceutical entry to the global master catalog	Admin Role Required

5.4 Security and Permissions Design

The application enforces a stateless security architecture utilizing JSON Web Tokens (JWT) passed securely via standard bearer authorization headers. Upon explicit session termination, token identifiers are written to a database-backed blocklist table, invalidating the token before its native expiration window closes.

System access is strictly controlled by explicit Role Guards. The Patient role is strictly isolated, granting access only to personal profile records, linked inventory items, private schedules, and individual notification feeds. Conversely, the Administrator role is restricted to global catalog updates, cross-layer system tracking, and manual override alerts, completely preventing access to patient medical logs.

Security Pillar Element	Programmatic Implementation Mechanics	System Deployment Context	Strategic Security Objective
Cryptographic Identity Verification	Issues signed, short-lived JWT payloads containing security claims	Intercepts requests across all protected system REST paths	Prevents token modification and session hijacking
Token Signature Blocklisting	Persists revoked token unique identifiers in high-speed storage	Triggers database verification checks during session routing	Inactivates sessions instantly upon user logouts
Role-Based Guard Access	Applies custom decorators to routes to verify role parameters	Enforces access boundaries across back-end endpoint layouts	Prevents unauthorized access across system profiles
Input Sanitization Guards	Applies strict schema models to clean incoming JSON payloads	Validates incoming parameters at the API gateway layer	Blocks code injection attacks and malformed database writes
Cross-Origin Isolation (CORS)	Configures whitelist rule fields targeting allowed web roots	Enforces browser-level request validation rules	Blocks cross-site requesting malicious behaviors

5.5 Backend Design for Scheduling and Notifications

The background processing system uses a multi-threaded execution ring to run asynchronous, non-blocking tasks. A high-priority scheduling loop runs continuously every minute to scan calendar records, verify intake windows, and generate real-time alerts. Concurrently, a lower-frequency inventory loop evaluates stock metrics, automatically dispatching restocking warnings when quantities cross lower safety boundaries.

Background Task Name	Execution Interval	Database Query Context	Resulting Operational Deliverable

Precision Scheduling Task	Every 60 seconds continuous loop	Scans active rules and checks current timestamps	Dispatches live intake alerts to target patient feeds
Inventory Auditing Task	Every 30 minutes periodic run	Evaluates stock counts against lower safety limits	Dispatches low-stock warnings to user notification hubs
revocation Auditing Task	Instant event-driven execution	Captures token hashes upon explicit logouts	Populates the token blocklist table to block old requests

Chapter 6: Execution Environment and Tools

This chapter details the production tools, server libraries, package configurations, project structures, and testing setups that form the platform's runtime environment. Documenting these requirements ensures that the application environment can be reliably duplicated, deployed, and maintained across different server infrastructures.

6.1 Software Tools and Libraries Used

The application runs on a mature, open-source stack designed for low-overhead web apps. It uses Flask as the primary application factory runner, backed by python-dotenv for secure environment configuration, SQLAlchemy for database transaction mapping, PyMySQL for database drivers, JWT headers for security, and APScheduler for background task execution loops.

Library Name	Release Version	Primary Runtime Role	Strategic Implementation Purpose
Flask	3.0.3	Primary web platform runner engine	Dispatches HTTP requests and mounts application packages
Flask-SQLAlchemy	3.1.1	Object-Relational Mapping data connector	Maps python entities to relational database rows cleanly
Flask-JWT-Extended	4.6.0	Cryptographic authorization manager	Handles stateless session signing and validation functions
Flask-Cors	4.0.1	Cross-Origin resource boundary controller	Enforces access rules for cross-layer browser requests
PyMySQL	1.1.1	Low-level database connection driver	Connects python runtimes to backend database engines natively
python-dotenv	1.0.1	Runtime environment parameter manager	Extracts production tokens and keys from isolated file assets

APScheduler	3.10.4	Multi-threaded task execution system	Drives the background processing ring and alert engines
Werkzeug	3.0.3	Security hashing and password utility provider	Handles secure, unidirectional data encryption for passwords
cryptography	42.0.8	Cryptographic algorithmic engine	Provides structural support for security protocols and token layers

6.2 Development Environment and Project Structure

The runtime environment boots from a centralized entry point (`run.py`), which calls the core application factory module (`create_app()`). The system configuration manager automatically loads environment variables, initializes database connection pools, configures cross-origin permissions, and boots the background task loops within a secure thread context upon application startup.

6.3 Programming Languages and Frameworks

The core application is built on Python, chosen for its stable, mature ecosystem of web tools, ORMs, task managers, and unit testing frameworks. The presentation layer uses standard vanilla JavaScript alongside clean HTML layouts and cohesive CSS themes to handle dynamic AJAX requests and display real-time dashboard notifications smoothly.

Language / Framework	Architecture Component	Project Deployment Context	Engineering Selection Rationale
Python	Back-End Stack	Drives core web services, repository interfaces, and test modules	Provides clean readability, high performance, and reliable packages
Flask	Server Layer	Handles application setup, routing tasks, and blueprint configurations	Lightweight design that supports clear layer separation
SQLAlchemy	Data Mapping Layer	Encapsulates database access rules inside python class objects	Decouples business logic from raw SQL query strings

JavaScript	Client UI Logic	Executes async API requests and handles real-time UI transitions	Enables reactive user dashboards without reloading pages
HTML / CSS	Presentation Layer	Defines the visual structures and responsive styling profiles	Maintains standard compliance across diverse device viewports

6.4 Database Server and Operating System

The persistence engine runs on a MySQL server instance configured with an absolute `utf8mb4` character profile to ensure perfect compatibility with complex multi-lingual string entries. The platform is cross-platform compliant, running reliably on both Linux server distributions and Windows development environments provided standard python runtimes, required packages, and target database connections are present.

Configuration Key	Runtime Target Setting	Functional Impact on System Operations
DB_HOST	localhost (Target Environment Default)	Defines the network connection location of the database engine
DB_PORT	3306 (Standard MySQL Gateway)	Defines the communication port for data traffic
DB_USER	Environment Custom Account Name	Handles identity tracking for database connections
DB_PASSWORD	Cryptographically Secure Password String	Blocks unauthorized access to raw database tables
DB_NAME	medication_system schema catalog	Isolates application records within a secure schema workspace
TRACK_MODIFICATIONS	False (Explicitly Terminated)	Disables unnecessary change tracking overhead to improve performance

6.5 Dependencies File

The platform maintains a highly disciplined `requirements.txt` file package manifest, ensuring total consistency across development, testing, and production environments.

Package Identifier	Primary Operational Functionality Description
Flask	Mounts the web framework, runs application factories, and handles routing tasks
Flask-SQLAlchemy	Manages database connection pools and handles object-relational abstraction layers
Flask-JWT-Extended	Enforces API token validation, claims parsing, and session guard blocks
Flask-Cors	Configures browser communication boundaries and isolates network routes
PyMySQL	Executes binary data streaming tasks targeting active database processes
python-dotenv	Injects sensitive production credentials into secure execution environments
APScheduler	Executes periodic, multi-threaded timing loops and schedules background alerts
Werkzeug	Computes cryptographic salt matrices to verify user access credentials safely
cryptography	Provides basic mathematical support for low-level server encryption tasks

6.6 Version Control and SCM Tools

The platform utilizes Git for rigorous version control, mapping code updates to a centralized repository structure. This allows development branches to be safely isolated, modifications to be tracked over time, and deployment builds to be rolled back instantly if an error occurs.

6.7 CI/CD Integration Tools

While the application avoids over-engineered cloud hosting setups, its modular design easily supports integration with automated CI/CD tools like GitHub Actions. This allows automated test scripts, code style checkers, and build validation suites to run automatically before code is merged into production branches.

6.8 Testing Environment

The testing infrastructure uses automated functional verification scripts (`tests/smoke_test.py`) to thoroughly validate core workflows: account creation, profile configuration, scheduling rules, alert routing, and intake confirmation. When an active MySQL instance is unavailable, the system automatically redirects transactions to an isolated in-memory SQLite runner to execute test workflows safely without affecting production data.

Evaluation Class	Validation Testing Tools	Target System Operations Evaluated	Expected Target Outcome
Smoke Testing	Automated Python verification scripts	Validates core end-to-end user workflows	Returns 100% successful execution codes
Syntax Verification	Integrated compiler syntax analysis tools	Scans application modules for formatting or structural flaws	Zero structural compilation anomalies
Schema Verification	MySQL Database comparison routines	Validates database mappings against python data models	Perfect alignment of data models and database tables
UI Architecture Checks	Browser testing tools	Verifies stylesheet linking and script asset loading paths	Flawless view rendering with zero missing asset errors

Chapter 7: Software Implementation

This chapter explains the actual programmatic construction of the platform. It details the system's file structure, module design, API route implementation, data repository layouts, business logic layers, front-end visual components, and the rule-based scheduling architecture that runs the core reminder engine.

7.1 File and Project Structure

The codebase follows an enterprise-grade multi-tiered project layout, strictly segregating structural packages to enforce the Separation of Concerns principle. This clean architectural boundary ensures that presentation templates, business logic services, and database repositories remain fully decoupled, allowing individual modules to be modified without breaking adjacent system layers.

7.2 Module Implementation

Core operations are encapsulated into distinct functional modules: `Auth` handles security onboarding and privilege tracking, `Medication` manages catalog variables, `Dose` oversees compliance auditing, and `Notification` coordinates alert delivery. Each module contains an isolated pair of service layers and routing blueprints, ensuring modular independence.

Module Name	Primary Programmatic Scope	Associated Project Code Assets	Core Functional Impact
Security Module	Onboarding, password hashing, and token issuance	auth_service.py, auth_routes.py package assets	Validates user identities and enforces security access roles
Catalog Module	Global master definitions and custom prescription configurations	medication_service.py, medication_routes.py assets	Manages available drug records and user vault inventories
Compliance Module	Intake verification logic, history tracking, and skipping actions	dose_service.py, dose_routes.py project codes	Audits therapy compliance and logs real-world patient outcomes
Alert Module	Real-time communication routing and message status updates	notification_service.py, notification_routes.py lines	Coordinates prompt pop-ups and updates dashboard read states
Worker Module	Multi-threaded scheduling loops and timing ring tasks	reminder_scheduler.py execution scripts	Automates alert production based on precise schedule

			milestones

7.3 API Implementation

API layers are built utilizing decoupled Flask Blueprints that parse incoming client parameters, validate input shapes, route execution payloads to business services, and return unified JSON objects via standardized success handlers. All sensitive routes use secure route guards to block unauthorized cross-role access attempts.

7.4 Data Access Layer Implementation

The data access layer strictly implements the Repository Pattern, completely isolating raw database interactions from the upper business logic modules. Specialized query functions encapsulate all complex relational lookups, filter conditions, and pagination offsets, ensuring database-agnostic business services.

Repository Class Name	Primary Data Scope	Key Implemented Query Methods	Target Data payload Return
BaseRepository	Global database fallback tasks	get_by_id, save, delete structural routines	Provides uniform CRUD access paths across all tables
DoseLogRepository	Historical compliance auditing	find_existing, get_history, count_by_status	Returns paginated compliance logs for physician audits
NotificationRepository	Alert dashboard management	get_for_patient, get_unread_for_patient	Returns lists of unread user alerts for real-time views
PatientMedicationRepository	User inventory monitoring	get_patient_medications, get_owned_patient_med	Provides filtered lists of personal vault inventories
DosageScheduleRepository	Prescription scheduling rules	get_all_active_by_patient_med	Returns active configuration parameters to scheduler loops

7.5 Business Logic Layer Implementation

The business service layer operates as the central coordination engine of the platform, executing the core operational workflows and transactional validation rules required by the system design. It contains strict validation checks to prevent invalid actions, updates database records within clean transactional boundaries, manages user privileges, and dynamically adjusts inventory counts upon confirmation.

Service Name	Primary Business Logic Focus	Enforced Operational Constraint Checks	Primary Transactional Outcome
AuthService	User authentication and session security	Blocks inactive users and rejects compromised account registrations	Signs secure access token signatures and verifies credentials
MedicationService	Inventory changes and scheduling rules	Prevents identical prescription entries from duplicating in user vaults	Commits new prescription models and links scheduling rules
DoseService	Intake confirmation and stock tracking	Blocks dose confirmation actions until the schedule window opens	Logs historical events and decreases physical stock counts
NotificationService	Alert life cycles and message read updates	Prevents duplicate alerts from firing for the same scheduling block	Tracks active system alerts and updates message read states

Code Logic Segment	Detailed Architectural Purpose
if require_notification and not has_dose_notification(...): raise ValidationError(...)	Prevents users from prematurely logging intake events before a schedule window opens
pat_med.current_quantity = available - required	Performs real-time database stock deductions immediately upon user validation
if current_quantity <= min_threshold: create_low_stock_notification(...)	Scans updated stock counts and automatically builds warning alerts if safety thresholds are breached
db.session.commit()	Saves multi-table data changes to database layers within a single atomic transaction

7.6 User Interface (UI) Implementation

The client layout utilizes a responsive presentation interface driven by explicit workspace modules (`login.html`, `dashboard.html`, `admin.html`). These workspaces connect to backend services via optimized JavaScript interaction scripts, utilizing non-blocking AJAX calls to update dashboards dynamically without requiring full page refreshes.

7.7 Reminder Mechanism and Smart Rule Logic

The platform implements a highly disciplined, deterministic rule-based scheduling logic instead of volatile, unvalidated predictive models. This ensures absolute reliability across the scheduling pipeline: the background task scans calendar boundaries, evaluates user recurrence rules, builds non-duplicate notification alerts, updates inventory counts, and generates historical compliance logs systematically.

Processing Phase	Programmatic Logic Action Performed	Resulting Operational Deliverable
Phase 1	Queries active calendar rules from the database storage layer	Identifies all medications requiring validation during the current day
Phase 2	Processes daily time matrices using internal validation trackers	Maps out specific scheduling slots that require alert generation
Phase 3	Scans active notification tables to ensure zero entry duplication	Eliminates duplicate alerts and blocks redundant notifications
Phase 4	Saves a new notification instance object to the active user dashboard	Displays an immediate pop-up alert over the active patient view
Phase 5	Deducts stock counts and writes new history rows upon user approval	Updates patient compliance logs and changes physical stock metrics
Phase 6	Evaluates lower-bound stock parameters and generates alert messages	Pushes automated replenishment warnings to user dashboards early

Chapter 8: Testing and Evaluation

This chapter outlines the verification methods used to validate the application's functionality, security, performance, and usability. It details the comprehensive test plan, multi-level testing approaches, detailed test cases, traceability metrics, quality KPIs, and user acceptance benchmarks required for formal academic validation.

8.1 Test Plan & Types of Tests

The test plan provides an extensive evaluation framework across three distinct layers. Unit tests evaluate individual service methods in complete isolation. Integration tests validate parameter passing and transaction tracking between routes, business logic, and repository layers. Finally, end-to-end system tests simulate real-world user workflows from front-end interface inputs down to persistent database storage records.

Testing Tier	Target System Focus	Evaluation Architecture Example	Strategic Validation Value
Unit Testing	Isolates and validates individual functions	validate_login(), confirm_intake() methods	Identifies logical programming flaws inside isolated modules
Integration Testing	Validates communication between structural layers	Route mapping -> service execution -> repository update	Ensures clean data passing and database transaction safety
System Testing	Validates end-to-end application workflows	Onboarding -> drug vault link -> schedule setup -> alert confirm	Confirms that the complete platform operates correctly for users
Acceptance Testing	Evaluates real-world requirement fulfillment	Auditing alert precision, stock limits, and UI responsiveness	Confirms the software meets all core user expectations
Regression Testing	Guards against code regressions during updates	Re-executing test suites after database schema updates	Ensures new changes do not break existing features

8.2 Test Diagrams and Detailed Test Cases

System validation is guided by explicit, highly detailed test specifications that link every functional requirement directly to verifiable test outcomes, ensuring thorough code coverage.

Test Case ID	Target System Workflow	Provided Input Parameters	Execution Step Sequence	Expected Functional Result	Priority Boundary
TC-01	User Account Onboarding	Valid names, unique emails, and strong passwords	Post registration data directly to account endpoint	Saves profile record and issues an access token payload	Critical Must-Have
TC-02	Secure Profile Login	Registered email and valid password strings	Post credential parameters directly to login endpoint	Validates payload and returns user session role claims	Critical Must-Have
TC-03	Inventory Record Generation	Valid drug profile code, stock values, and safety limits	Post inventory payload to the vault endpoint path	Saves a new tracking record inside the inventory table	Critical Must-Have
TC-04	Schedule Configuration	Active inventory link code, day maps, and target time inputs	Post scheduling matrices to the calendar endpoint	Persists scheduling objects and logs daily intake windows	Critical Must-Have
TC-05	Intake Event Tracking	Active alert identifier and current calendar time parameters	Post intake confirmation data to compliance endpoints	Saves history entry, deletes alert, and decreases stock	Critical Must-Have
TC-06	Skipped Intake Auditing	Active alert identifier mapping to a valid pending	Post skipping payload directly to compliance endpoints	Writes a skipped status entry into the history logs	Standard Should-Have

		notification			
TC-07	Lower stock Alerts	Manual modification of stock values down to zero	Trigger inventory evaluation routines on backend	Background task automatically dispatches a stock alert	Critical Must-Have
TC-08	Authorization Rejection	HTTP request containing completely missing token lines	Attempt access to a protected system API route	Gateway layer blocks request with an HTTP 401 error	Critical Must-Have

8.3 Test Results and Failure Analysis

System evaluation focuses heavily on identifying potential failure points within complex scheduling or notification workflows, ensuring the platform implements graceful error handling and robust recovery mechanisms.

Failure Scenario	Potential Technical Root Cause	Direct Operational Impact	Engineered Resolution Strategy
Premature Intake Logging	User attempts to log an intake event before the schedule window opens	Blocks request to protect log accuracy	Returns an explicit, clear validation message to the client
Scheduler Duplication	A timing loop fires repeatedly, creating duplicate notifications	Clutters the user dashboard with redundant alerts	Enforces rigid database constraints to block duplicate rows
Incomplete Data Entry	A form is submitted with missing parameters or empty fields	Causes storage transactions to fail	Enforces strict client-side validation rules before transmission
Session Expiration	An API request utilizes an expired or corrupted session token	Blocks backend data access	Redirects the user to the login screen and prompts a session renewal

8.4 Quality Metrics

Objective quality metrics guide the platform's evaluation framework, establishing clear benchmark targets for functional correctness, system reliability, interface usability, API performance, and security.

Quality Indicator KPI	Technical Quality Metric Definition	Target Performance Benchmark	Operational Project Priority
Code Test Coverage	The percentage of critical code branches verified by scripts	100% execution across all critical operational logic	High-Priority Benchmark
Defect Density Score	The volume of programming flaws relative to module sizes	Maintained at absolute minimums prior to build releases	High-Priority Benchmark
API Response Latency	The time elapsed from issuing a request to receiving a reply	Standard transactions execute in under 200 milliseconds	High-Priority Benchmark
Interface Usability	The usability score tracking user workflow efficiency	Clean, accessible interfaces requiring minimal user steps	Medium-Priority Benchmark
Platform Reliability	The stability score of scheduler tasks and notification delivery	100% accurate alert generation with zero timing drifts	High-Priority Benchmark

8.5 User Acceptance Testing (UAT)

User acceptance parameters ensure the application provides a smooth, accessible experience for patients, evaluating workflow clarity, inventory management, notification formatting, and compliance tracking.

UAT Evaluation Category	Target Usability Metric Surveyed	Target User Score Range	Technical UX Evaluation Focus
Layout Accessibility	Was the interface design easy to understand and navigate?	Target baseline of 4.5 out of 5.0	Evaluates visual hierarchy and theme consistency

Inventory Linking	Were the steps to add a medication and set stock limits simple?	Target baseline of 4.8 out of 5.0	Evaluates input form validation and processing steps
Notification Formatting	Was the alert message clear when a medication was due?	Target baseline of 4.7 out of 5.0	Evaluates message presentation and action buttons
Compliance Tracking	Was reviewing past medication history straightforward?	Target baseline of 4.6 out of 5.0	Evaluates log presentation and data sorting layouts

Chapter 9: Results and Analysis

This chapter presents the system's operational outcomes. It analyzes feature execution, evaluates project objective fulfillment, benchmarks performance metrics against industry alternatives, and provides an objective review of architectural strengths and functional limitations.

9.1 System Deployment Results

The implemented codebase achieves a highly reliable operational lifecycle. Account authentication separates access roles flawlessly, medication vault tools link records cleanly, scheduling matrices map out complex timing rules accurately, background threads dispatch alerts on time, and inventory updates adapt instantly upon user validation.

Core Operational Domain	Actual Implemented System Result	Direct Patient/Operator Impact	Technical Architectural Context
Session Verification	Separate patient and admin access routes secured via cryptographic tokens	Protects user data privacy and isolates system roles	Driven by cryptographic claims parsing
Inventory Auditing	Enables adding, changing, pausing, and activating vault records	Grants patients full control over their active drug lists	Maintained via specific data models
Schedule Management	Supports creating multi-tiered day and time scheduling matrices	Organizes complex treatment routines across clear timelines	Structured via relational schema rules
Alert Delivery	Dispatches real-time intake pop-ups and low-stock warning notifications	Reduces missed medication events and prevents supply gaps	Prevents duplicate alerts dynamically
Compliance Logging	Maintains unalterable ledger entries for all intake behaviors	Provides clean historical compliance data for clinical audits	Saves permanent records to history tables
System Management	Provides dedicated control consoles for global catalog maintenance	Ensures clean catalog management without data mixing	Restricted to authorized admin roles

9.2 Comparative Analysis with Existing Systems

Quantitatively benchmarking the platform highlights its architectural advantages over traditional alternatives, demonstrating the clear value of a single, unified codebase.

Functional Capability Evaluation	Standard Alert Software	Dedicated MERM Devices	Illness-Specific Tools	Engineered Project System
Automated Time Reminders	Fully Functional Implementation	Fully Functional Implementation	Fully Functional Implementation	Fully Functional Implementation
Unalterable History Logging	Incomplete or Highly Manual	Fully Functional Implementation	Fully Functional Implementation	Fully Functional Implementation
Live Inventory Auditing	Completely Unsupported	Completely Unsupported	Incomplete and Restrictive	Fully Functional Implementation
Multi-Role Management Panels	Completely Unsupported	Completely Unsupported	Completely Unsupported	Fully Functional Implementation
Cryptographic Session Security	Rudimentary or Omitted	Completely Unsupported	Incomplete and Stateful	Fully Functional Implementation
Deterministic Task Scheduling	Fully Functional Implementation	Fully Functional Implementation	Incomplete and Restrictive	Fully Functional Implementation

9.3 Objective Fulfillment Extent

Reviewing the core project objectives defined in Chapter 1 confirms that the system completely fulfills its design goals within a secure, stable, and highly maintainable software architecture.

Design Objective Target	Real-world Project Implementation Status	Direct Technical System Impact	Fulfillment Level Evaluation
Integrated Web Platform	Fully implemented web infrastructure running smoothly	Provides end-to-end execution across all core layers	100% Full Objective Fulfilling

Precision Time Reminders	Background worker threads calculate intake slots accurately	Reduces missed medication events significantly	100% Full Objective Fulfilling
Inventory Stock Limits	Automated stock tracking triggers warnings at boundary limits	Provides proactive, early alerts before supplies deplete	100% Full Objective Fulfilling
Compliance Ledger Auditing	Unalterable logging tables record all user intake behaviors	Enables precision historical data auditing for clinicians	100% Full Objective Fulfilling
Role Privilege Isolation	Cryptographic guards separate user profiles from admin tools	Enforces strict data boundaries and access security	100% Full Objective Fulfilling

9.4 Strengths and Weaknesses Analysis

An honest, objective review of the platform highlights its main architectural strengths alongside its practical deployment limitations.

Architectural Strength Profiles	System Limitation / Future Expansion Targets
Highly decoupled Layered Architecture simplifies long-term maintenance	Lacks integrated numerical performance dashboards in production environments
Strict cryptographic Role Guards ensure robust access security boundaries	Requires manual inventory count inputs from users during initial setup
Deterministic backend task loops calculate scheduling slots flawlessly	Lacks predictive machine learning models or telemetry integrations
Integrated inventory tracking generates automated low-stock alerts cleanly	Operates as an independent web application, lacking native mobile app shells

9.5 Performance and Quality Indicators

An analysis of the platform's quality KPIs confirms exceptional technical scores across system correctness, design completeness, feature traceability, long-term maintainability, layout usability, and data security.

Chapter 10: Conclusion and Recommendations

This final chapter concludes the project report. It summarizes the core software engineering achievements, reviews the technical challenges managed during development, maps out future system enhancement paths, and provides practical recommendations for subsequent engineering teams.

10.1 Project Summary and Main Achievements

This project was engineered to solve a critical, real-world healthcare challenge: the high clinical risk of patient non-adherence due to complex medication routines. The resulting platform provides a secure, fully integrated web application built on top of a decoupled, maintainable multi-tiered architecture.

The primary milestone achieved is the creation of an automated end-to-end data pipeline. The application successfully connects global master catalogs to user-specific vaults, links complex day/time scheduling matrices, handles multi-threaded background alert routing, prompts real-time interface notifications, processes instant stock deductions upon user validation, and logs unalterable compliance histories seamlessly.

Core Project Milestone	Technical Implementation Outcome	Direct System Value Delivered	Maturity Level Assessment
Stateless Security	JWT authentication routing with token blocklist tracking	Secures user profiles and blocks unauthorized data requests	Production-Ready Deployment State
Inventory Management	Centralized pharmaceutical indices mapped to private drug vaults	Isolates master catalog definitions from private user data	Production-Ready Deployment State
Precision Scheduling	Flexible scheduling rules managing multi-timed daily matrices	Organizes complex treatment tracking across clean timelines	Production-Ready Deployment State
Alert Infrastructure	Background worker threads running automated alert check loops	Minimizes medication oversights and tracks supplies	Production-Ready Deployment State
Compliance Auditing	Unalterable history logging tracking user intake behaviors	Provides clean historical compliance records for clinical audits	Production-Ready Deployment State

Administrative Control	Dedicated management consoles with global database filters	Enforces global system auditing without data mixing	Production-Ready Deployment State

10.2 Challenges and Difficulties Faced

The primary software design challenge centered on maintaining perfect sync across interconnected data assets during concurrent transactions. For example, updating an inventory item required updating schedule models, adjusting background task execution chains, calculating timezone offsets, and updating dashboard states simultaneously without creating data race conditions.

A second major challenge involved maintaining absolute layer separation across the multi-tiered architecture. Decoupling database entities, repository methods, business services, API endpoints, and view controllers demanded disciplined package naming conventions, consistent payload shapes, and standardized error handlers to keep the codebase clean and maintainable.

Identified Technical Challenge	Operational System Threat	Engineered Resolution Strategy	Resulting Structural Outcome
Data Chain Synchronization	Potential transaction drifts between stock levels and histories	Enforced strict validation checks before committing changes	Guaranteed database transaction safety
Alert Duplication Loops	Background task schedulers firing duplicate notification pop-ups	Enforced rigid unique indices inside database tables	Eliminated redundant client-side alerts
Cross-Layer Exploits	Unauthorized route access or privilege escalation attempts	Fortified backend paths with custom role decorators	Enforced rock-solid security boundaries
Response Fragmentation	Malformed or broken front-end view layouts	Standardized server replies using a uniform JSON handler	Ensured clean, reliable interface rendering

10.3 Future Development Proposals

While the current release completely fulfills its initial project goals, its clean modular layout easily supports future functional expansions, such as advanced data visualization tools, native mobile wrappers, smart third-party notifications, and secure clinical sharing links.

Proposed Extension Domain	Technical Enhancement Feature	Direct Operational Value Add	Development Priority Target
Compliance Analytics	Interactive graphical compliance histories for physicians	Provides clear visual overviews of compliance trends	High-Priority Future Phase
Mobile Access Shells	Progressive Web App (PWA) wrapper architecture installation	Enables native offline access from mobile phone devices	High-Priority Future Phase
Smart Notifications	Third-party communication integrations	Sends immediate alternative text alerts if web views are closed	High-Priority Future Phase
Enterprise Integration	OAuth2 single sign-on integration	Supports secure integration with institutional clinic portals	Medium-Priority Future Phase
Data Reliability	Automated system backup routines and clinical report creation	Enables rapid data retrieval and reliable records sharing	High-Priority Future Phase

10.4 Practical Recommendations

Subsequent software engineering teams extending this codebase are strongly advised to prioritize backend layer stability before adding cosmetic front-end features. Future modifications should follow a disciplined design-first, test-driven workflow, ensuring new code additions do not disrupt the system's precise timezone calculations or relational integrity rules.

Target Audience Group	Strategic Technical Recommendation	Intended Architectural Outcome
Engineering Students	Keep service layers small and document new API routes early	Prevents code clutter and speeds up development
Academic Supervisors	Focus code evaluations on core scheduling safety constraints	Ensures high-quality, reliable project implementations

Subsequent Project Teams	Add interactive charts and comprehensive compliance dashboards	Significantly increases real-world clinical utility
Deployment Operators	Audit security configurations and token lifespan profiles before hosting	Guarantees production-grade data privacy and safety

10.5 Final Conclusion

This project successfully demonstrates that a centralized web application built on top of a highly disciplined, layered architecture provides a highly effective software solution for medication management and adherence tracking. By unifying automated inventory auditing, deterministic schedule mapping, multi-threaded alert processing, and immutable compliance logging, the platform delivers a robust, scalable system that provides clear technical blueprints for subsequent software engineering endeavors.